

SYSTEM ARCHITECTURE

Overall Solution Architecture:

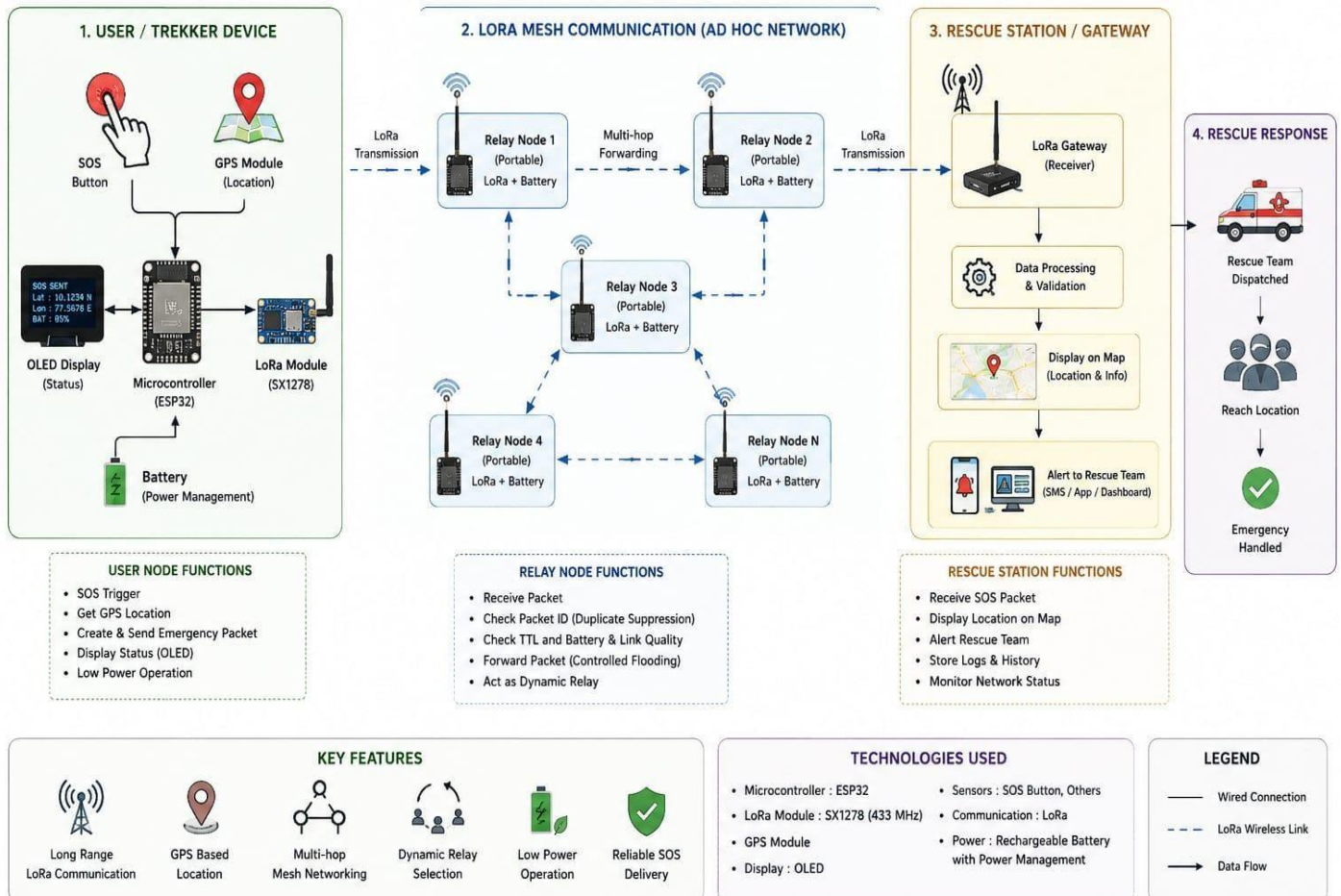
The TerraLink system follows a **multi-stage emergency communication architecture**:
Emergency Trigger → **Data Acquisition** → **SOS Packet Formation** → **LoRa Transmission** → **Relay Nodes** → **Rescue Station**

1. **Data Acquisition:** The touch sensor detects the emergency trigger, while the GPS module provides the user's location.
2. **Data Processing:** The microcontroller processes the inputs and creates an SOS packet containing the emergency and location information.
3. **Wireless Transmission:** The SOS packet is transmitted through the LoRa module.
4. **Packet Relay:** Intermediate TerraLink relay nodes receive and forward the packet until it reaches the destination.
5. **Rescue Station:** The final node receives the emergency message and location data, enabling a quick rescue response.

This architecture enables **long-range, reliable, and infrastructure-independent emergency communication**.

TERRALINK – OVERALL SOLUTION ARCHITECTURE

LoRa Based Emergency Communication System with GPS and Dynamic Relay



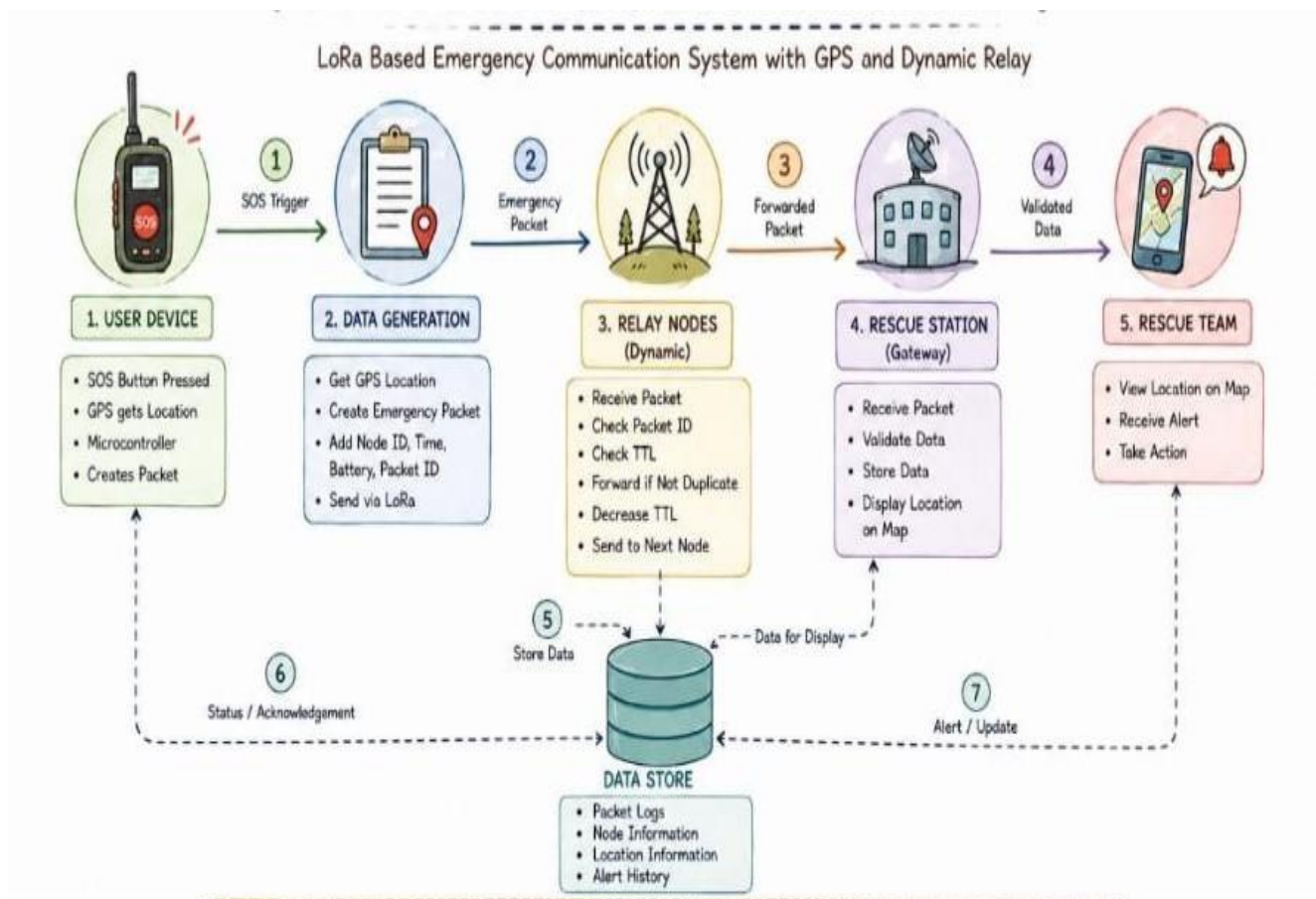
DATAFLOW DIAGRAM:

The TerraLink Data Flow Diagram represents the flow of information between the user, hardware modules, software, and communication network.

Data Flow:

User → Touch Sensor → Microcontroller → GPS Data + SOS Packet → LoRa Module → Remote TerraLink Node

The microcontroller processes the touch input, collects GPS coordinates, creates the SOS packet, and transmits it through LoRa. The received emergency information is processed and displayed on the OLED display.



HARDWARE–SOFTWARE PARTITIONING

The TerraLink system is divided into hardware and software components based on their respective functions.

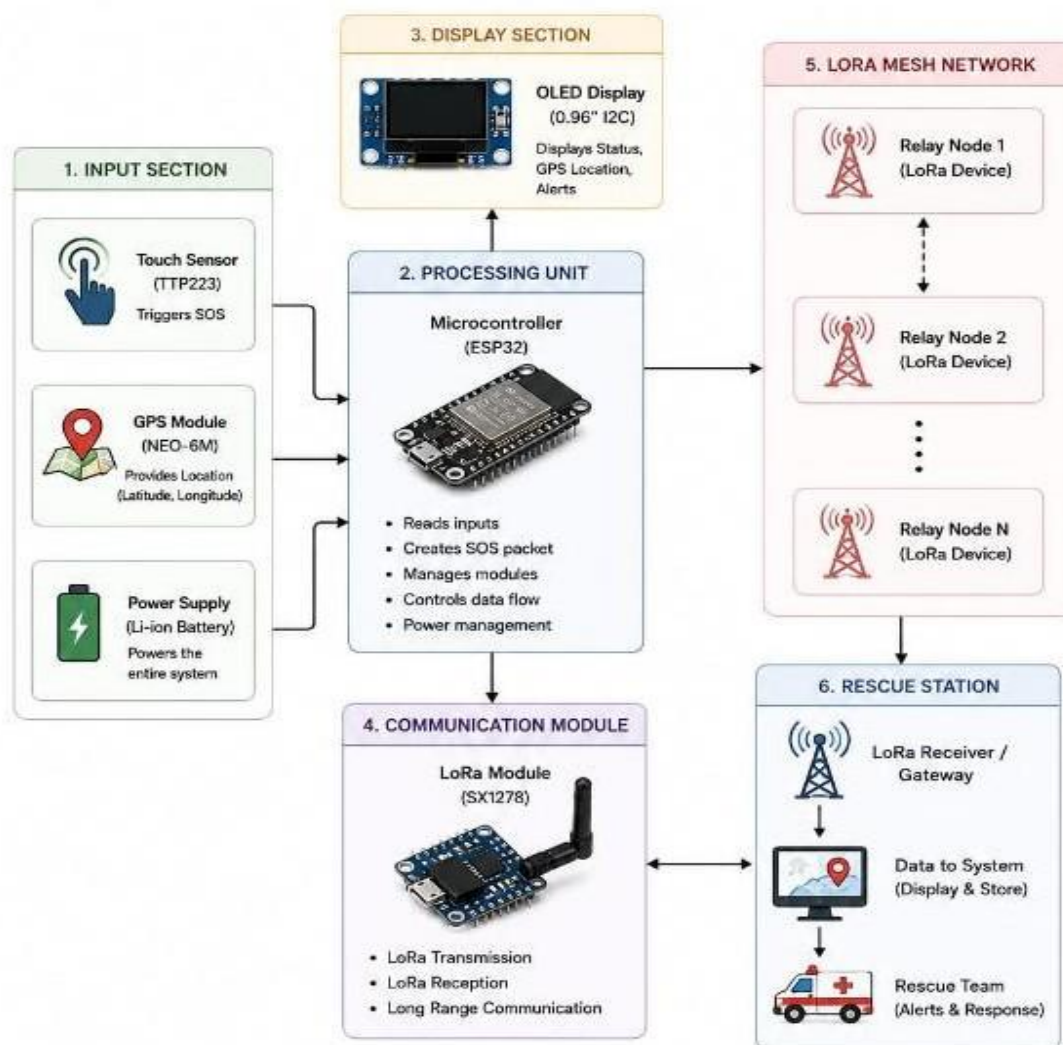
- **Hardware:** Includes the microcontroller, LoRa module, GPS module, touch sensor, and OLED display for processing, communication, location tracking, user input, and information display.
- **Software:** Handles sensor input processing, GPS data acquisition, SOS packet generation, LoRa communication, and OLED display management.

This partitioning ensures efficient, reliable, and coordinated operation of the TerraLink emergency communication system

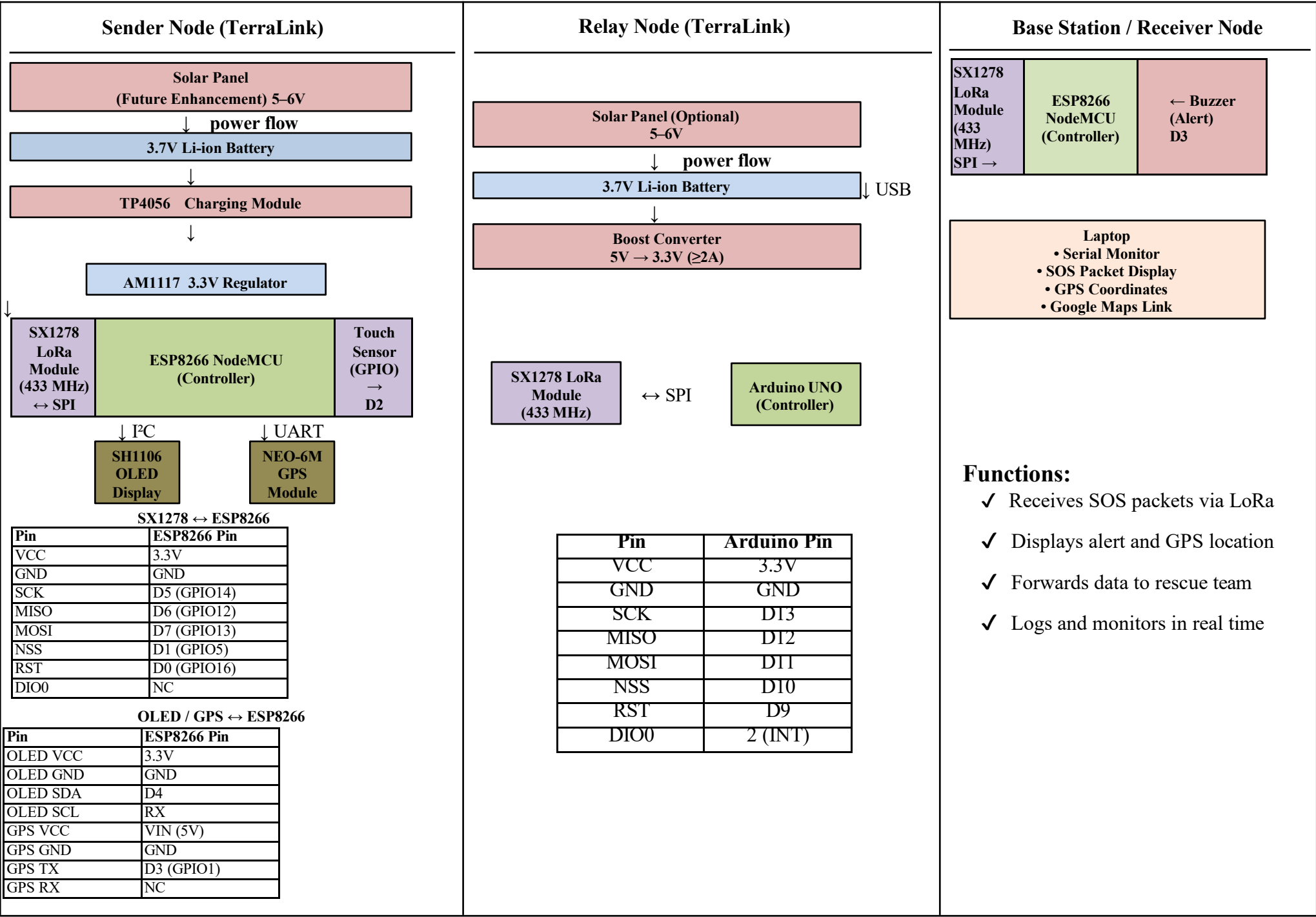
HARDWARE COMPONENTS			SOFTWARE MODULES		
Component	Image	Function	Module	Image / Icon	Function
Microcontroller (ESP32)		Controls all operations, processes data and manages communication.	1. Device Management		• Initialize hardware • Device ID management • Battery monitoring
LoRa Module (SX1278)		Provides long range wireless communication.	2. Sensor Management		• Read touch sensor • Detect SOS trigger • Trigger events
GPS Module (NEO-6M)		Provides location information (latitude, longitude).	3. GPS Management		• Acquire location • Parse GPS data • Provide coordinates
Touch Sensor (TTP223)		User triggers SOS alert using touch.	4. Communication (LoRa Stack)		• Create packet • LoRa transmission • Packet reception
OLED Display (0.96" I2C)		Displays status, GPS location and alerts.	5. Relay Management		• Check packet ID • TTL handling • Forward packet • Duplicate suppression
Power Supply (LI-Ion Battery)		Provides power to the entire system.	6. Data Management		• Store logs • Store location • Store alerts
Antenna (433 MHz)		Enhances LoRa signal transmission.	7. User Interface		• Display status on OLED • Show location • Show alerts
			8. Power Management		• Sleep / Deep sleep modes • LoRa power control • Optimize power usage

FUNTIONAL BLOCKDIAGRAM:

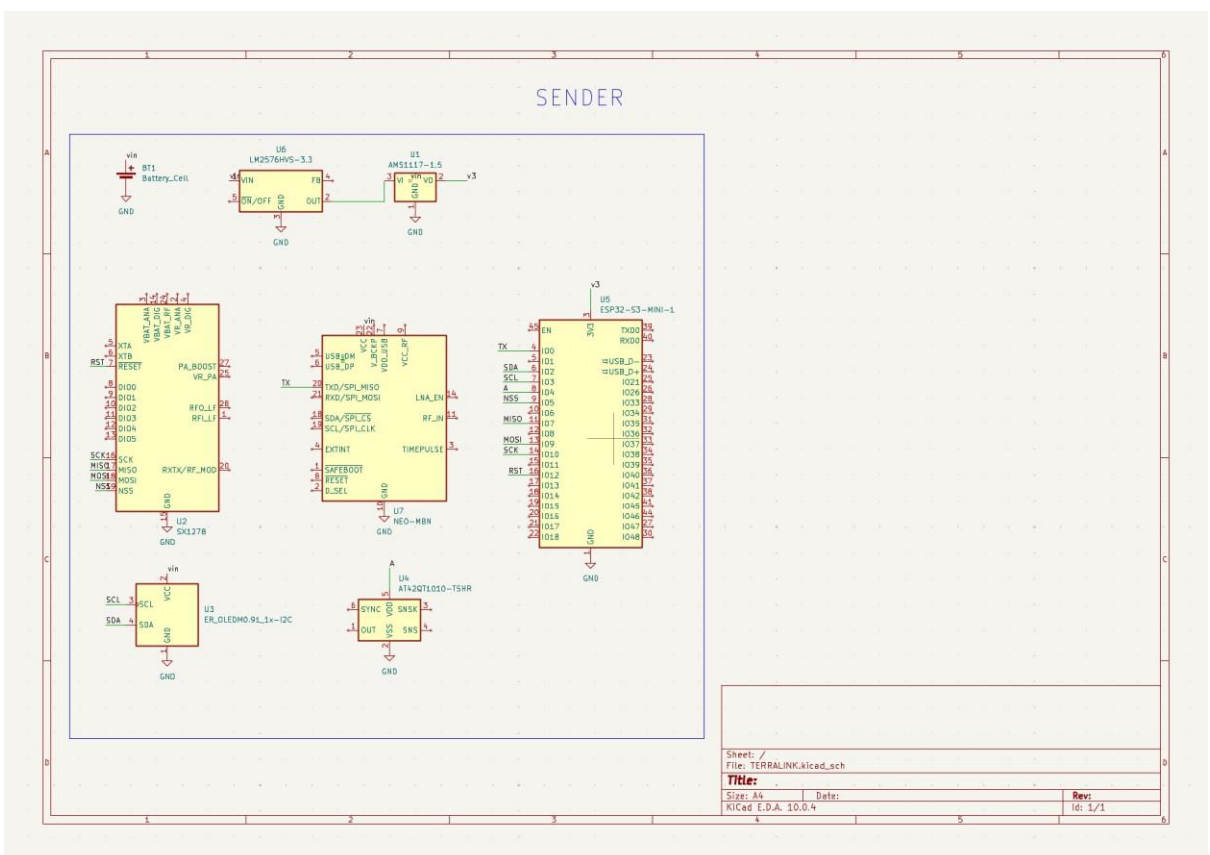
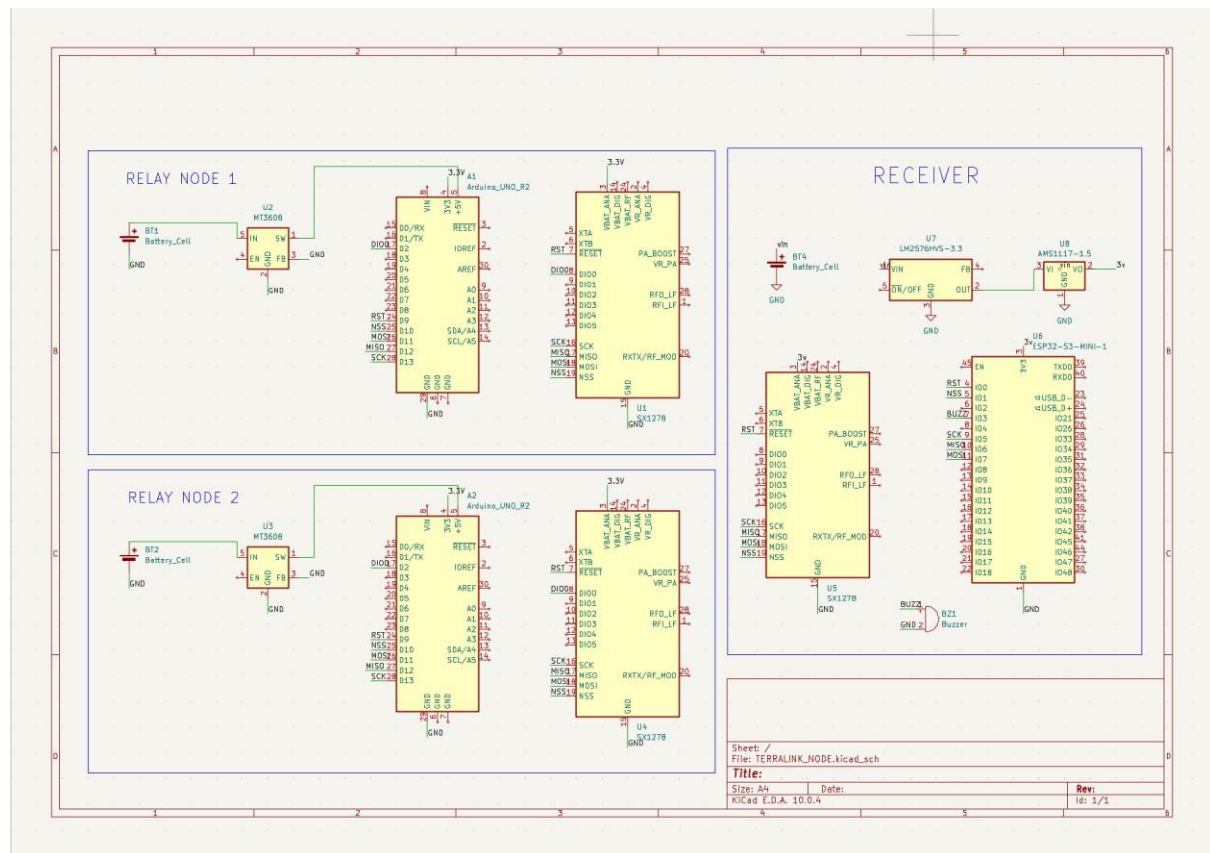
- **Emergency Detection:** Touch sensor detects an emergency.
- **Data Processing:** Microcontroller processes the emergency signal.
- **Location Tracking:** GPS obtains the user's location.
- **LoRa Transmission:** Emergency message and location are transmitted through LoRa.
- **Relay Nodes:** Relay nodes forward the message to the destination.
- **OLED Display:** OLED displays system status and location information.



HARDWARE DESIGN



CIRCUIT SCHEMATIC:



BILL SUMMARY:

Project Title : TERRALINK

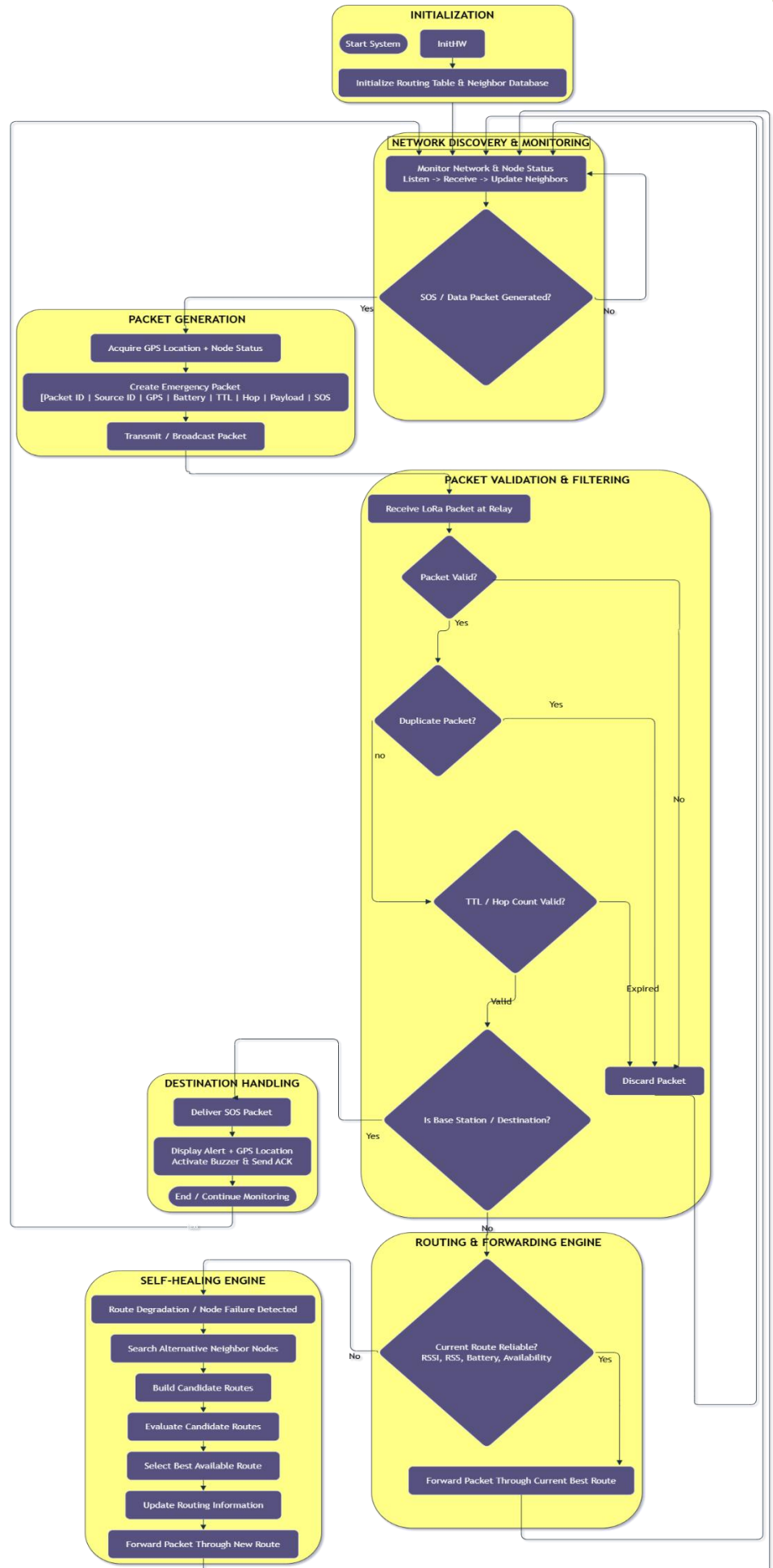
Certified that the following expenditures are made by us and the bills are admitted for the above project only. Bills are in order. It is assured that, no other claims are made in these bills.

Sl.No.	Bill No.	Bill Date	Description	Amount
1.	MZERD26 -27/1930	07-05-2026	Big Push Button Switch, AMS1117-3.3V, MT3608 Boost Converter	₹ 228.00
2.	MZERD26 -27/1930	07-05-2026	TP4056 Module, 3.7V Li-ion Battery, NEO-6M GPS, AI Thinker LoRa SX1278	₹ 2696.00
3.	MZERD26 -27/1930	07-05-2026	18650 Single Cell Holder	₹ 117.00
4.	MZERD26 -27/1930	07-05-2026	ESP8266 CP2102, Breadboard, 4.5V Mini Solar Panel	₹ 669.00
5.	MZERD26 -27/1930	13-05-2026	TTP223 Touch Module, ARDUINO UNO R3, Wireless Antenna, 16×2 LCD with I2C	₹ 999.00
6.	MZERD26 -27/1930	13-05-2026	Breadboard GL-12	₹ 91.50
7.	MZERD26 -27/1930	28-05-2026	Arduino UNO R3	₹ 496.50
8.	MZERD26 -27/1930	21-07-2026	0.96-inch OLED Display	₹ 284.00
TOTAL				₹ 5581.00

(Rupees FIVE THOUSAND FIVE HUNDREAD AND EIGHTY-ONE Only)

SOFTWARE DESIGN

FLOW CHART:



ALGORITHM:

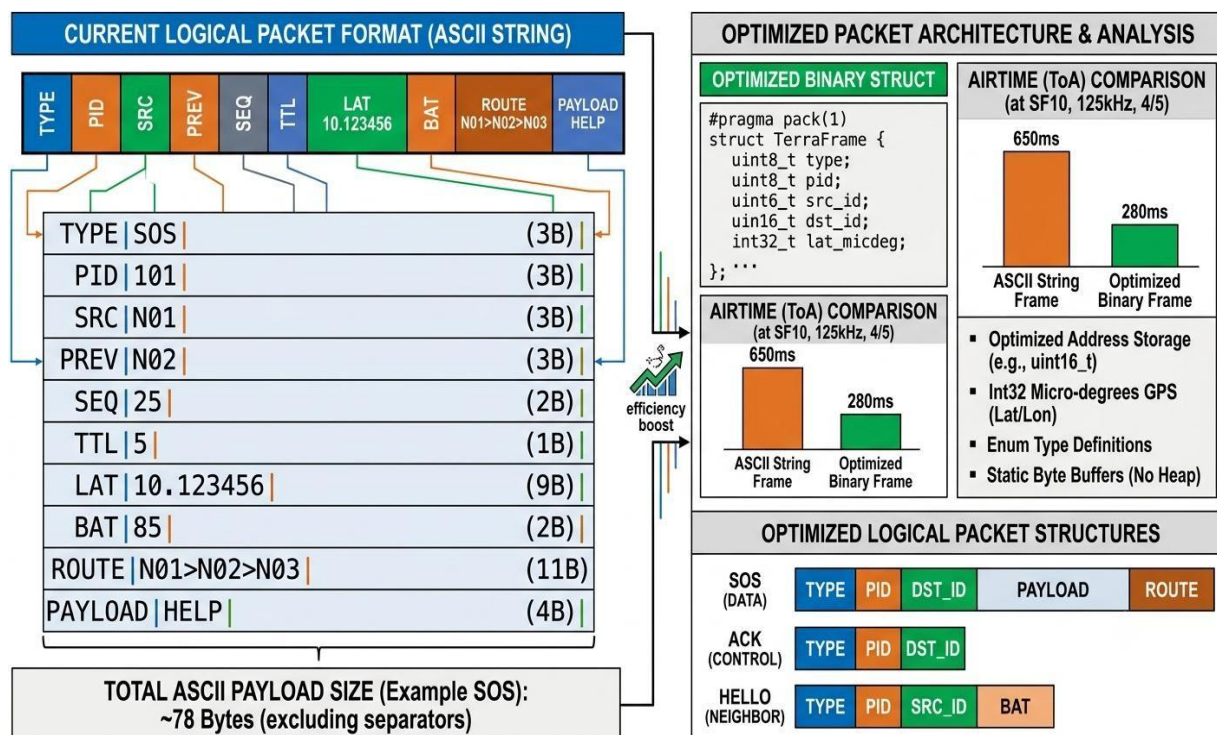
1. START
 2. Initialize MCU, LoRa, GPS, SOS input, display, and network tables.
 3. Check **LoRa initialization**.
 - If **NO**, retry initialization.
 - If **YES**, continue.
 4. Continuously monitor nodes, links, and SOS input.
 5. If **SOS = YES**, acquire GPS data.
 6. Form the packet with **Source ID, Packet ID, GPS, Battery, SOS, TTL, Hop Count, and Destination**.
 7. Discover and evaluate available neighbouring nodes.
 8. Select a suitable **next hop**.
 - If no next hop is available, wait/retry neighbour discovery.
 9. Check **TTL**.
 - If expired, **drop packet**.
 - Otherwise, decrement TTL and increment Hop Count.
 10. Transmit the packet to the selected next hop.
 11. At the receiving node, validate the packet.
If invalid, **drop packet**.
 12. Check for duplicate using **Source ID + Packet ID**.
If duplicate, **drop packet**.
 13. Check whether the **destination is reached**.
 - ✓ If **YES**, deliver packet to the Base Station.
 - ✓ If **NO**, continue forwarding.
 14. Check node/link status.
If the current next hop has failed, mark it **unavailable**.
 15. Discover an alternative neighbour and **recalculate the next hop**.
 16. Forward the packet through the new route.
 17. At the Base Station, display emergency information and activate the alert.
 18. Return to **continuous monitoring**.
 19. STOP / Repeat continuously.
-

COMMUNICATION PROTOCOL

The Dynamic TerraLink Communication Protocol defines how an SOS packet is created, routed, validated, forwarded, acknowledged, and recovered when a communication link fails. It uses lightweight AODV-inspired routing, duplicate filtering, TTL-based loop prevention, hop-by-hop ACK, retransmission, and self-healing.

1. Packet Creation and Packet Structure

When an End Node detects an SOS event, it creates a structured packet containing the source, destination, sequence number, TTL, location, battery status, and emergency payload.



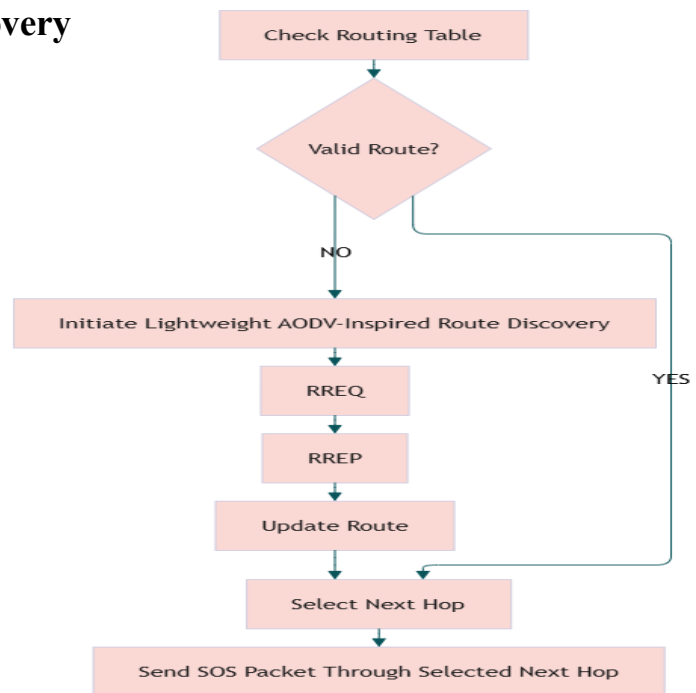
The main fields are:

- **TYPE** – Identifies the packet type.
- **PID** – Packet identifier.
- **SRC** – Source node.
- **DST** – Destination/base station.
- **SEQ** – Sequence number used for packet identification and duplicate filtering.
- **TTL** – Limits packet lifetime.
- **GPS** – Emergency location.
- **BATTERY** – Source-node battery status.
- **PAYLOAD** – Emergency information.

2. Route Selection and Route Discovery

Before transmitting an SOS packet, the node checks its routing table.

TerraLink uses a lightweight AODV-inspired route discovery mechanism rather than claiming full standard AODV implementation.

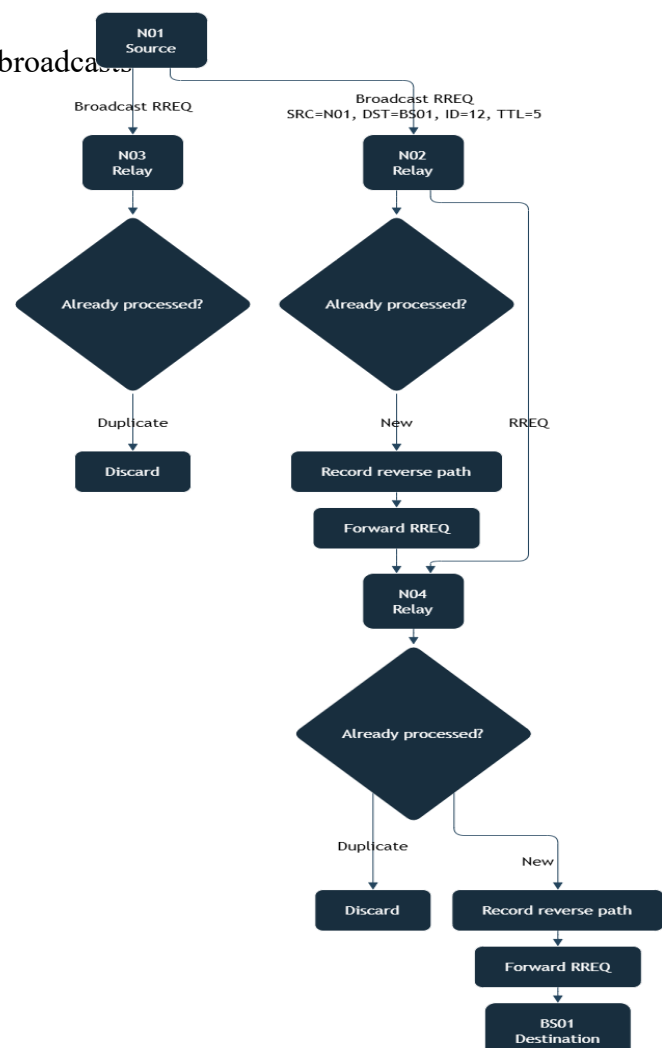


3. RREQ — Route Request

When there is no valid route, the source broadcasts an **RREQ**.

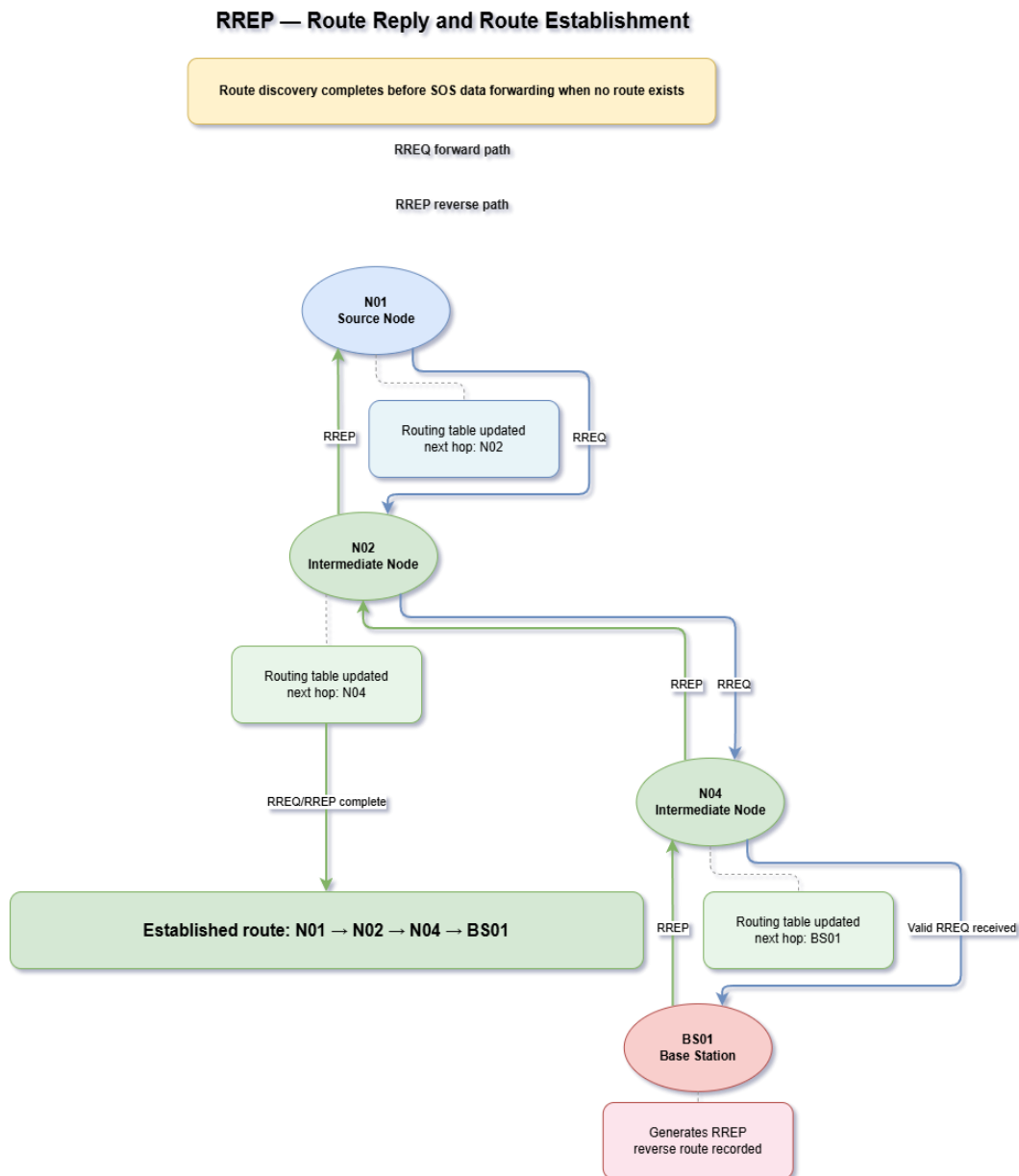
A relay receiving the RREQ:

1. Checks whether this RREQ has already been processed.
2. If duplicate → discard.
3. If new → record the reverse path.
4. Forward the RREQ toward the destination.



4. RREP — Route Reply

When the Base Station receives a valid RREQ, it generates an RREP. The RREP travels back through the reverse path:



The important point is that **RREQ/RREP** establishes the route before the SOS data is forwarded when no route exists.

5. Duplicate Filtering

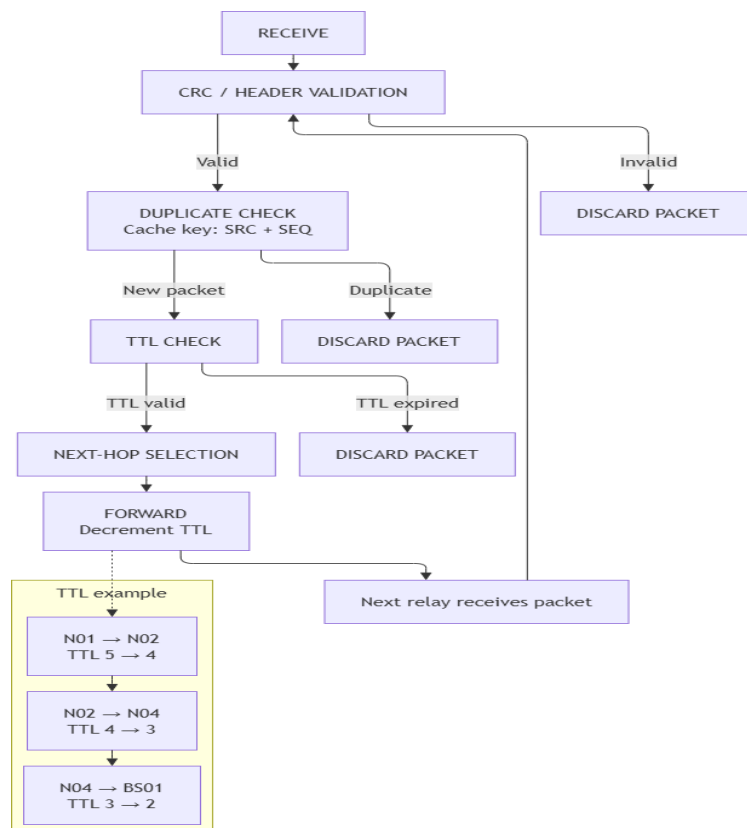
Each node maintains a small SRAM cache using:

SRC + SEQ

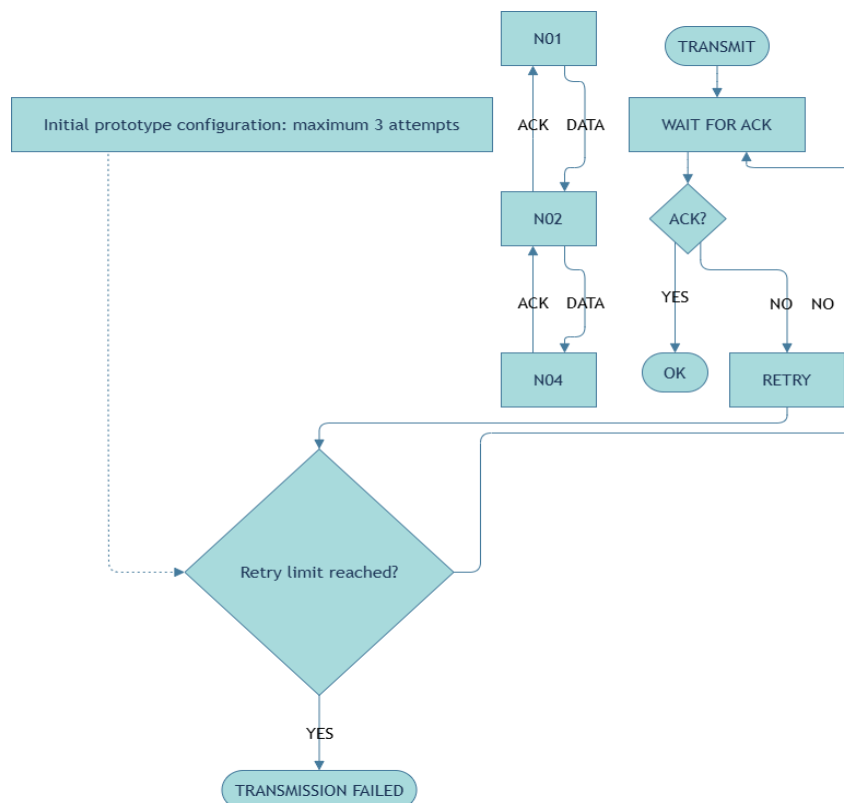
If the same source/sequence combination has already been processed, the packet is discarded to prevent repeated forwarding.

6. Packet Validation and Forwarding

Every relay validates a received packet before forwarding it. The processing order is:



7. Hop-by-Hop ACK and Retransmission

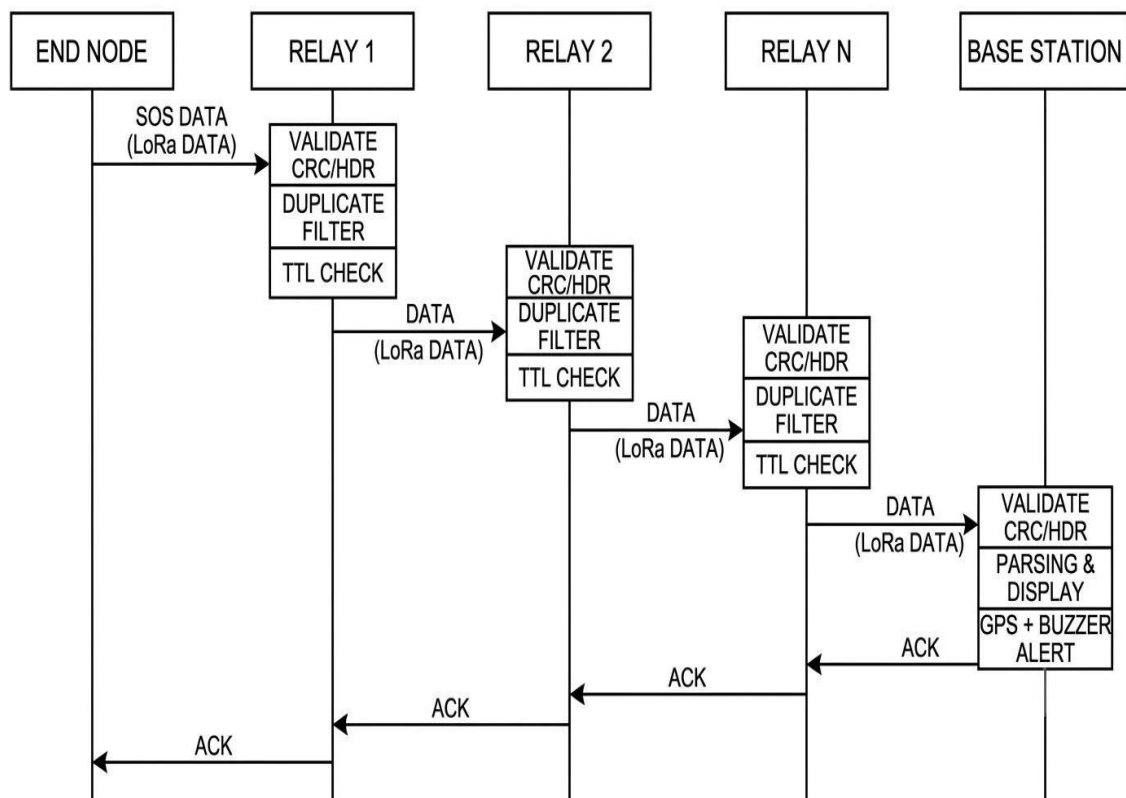
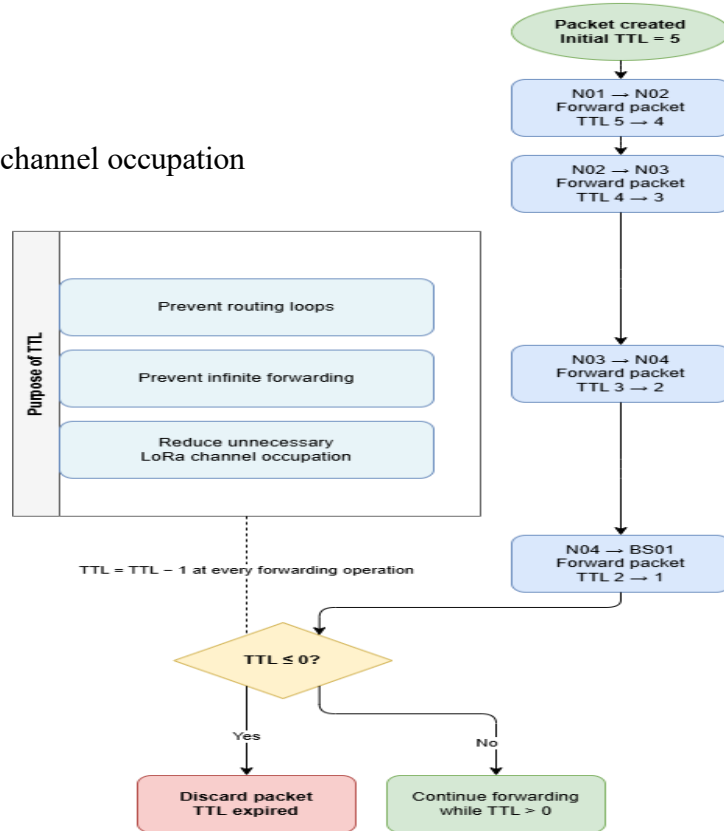


8. TTL — Packet Lifetime

TTL limits how many forwarding operations a packet can undergo.

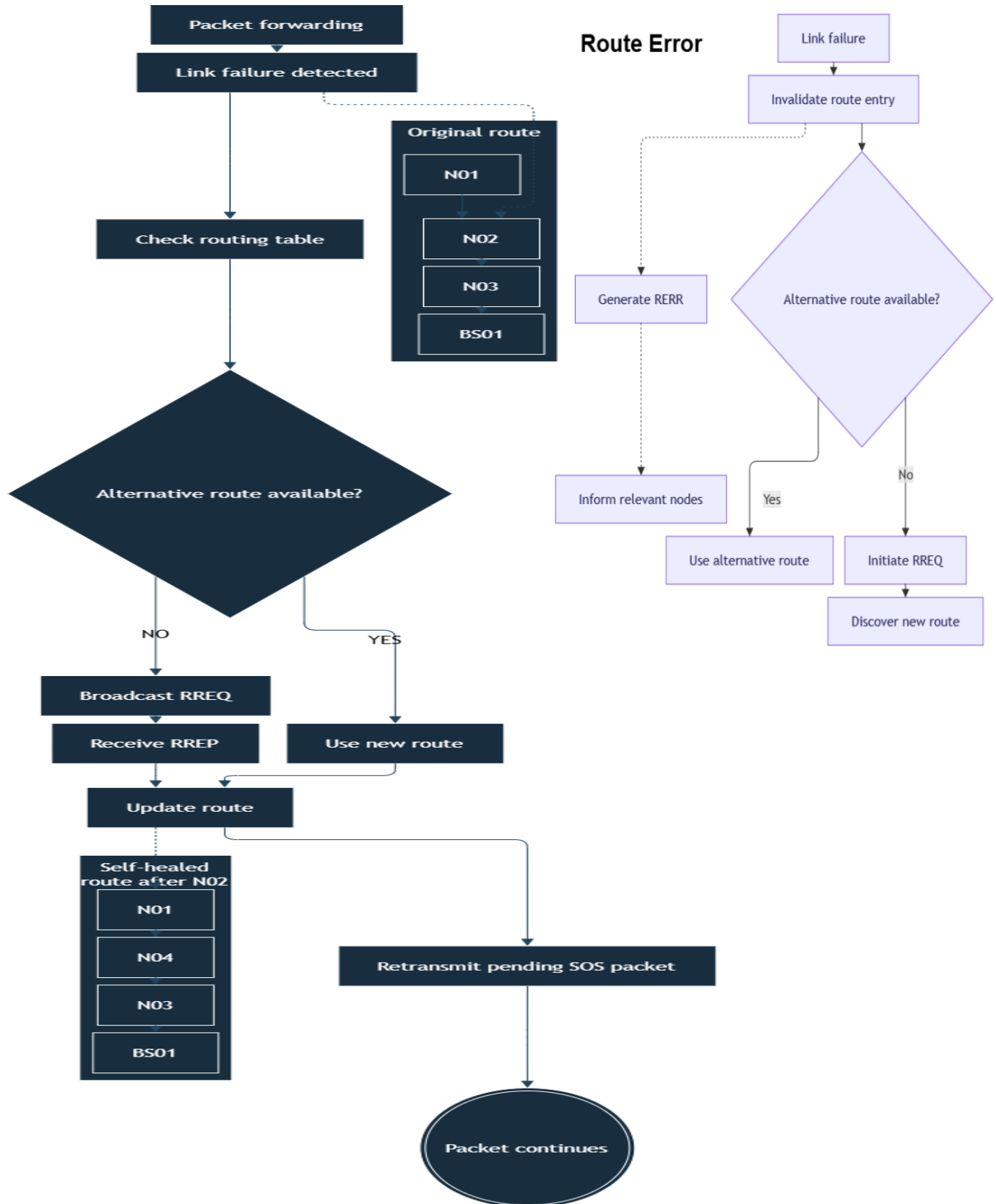
The purpose is to prevent:

- Routing loops
- Infinite forwarding
- Unnecessary LoRa channel occupation



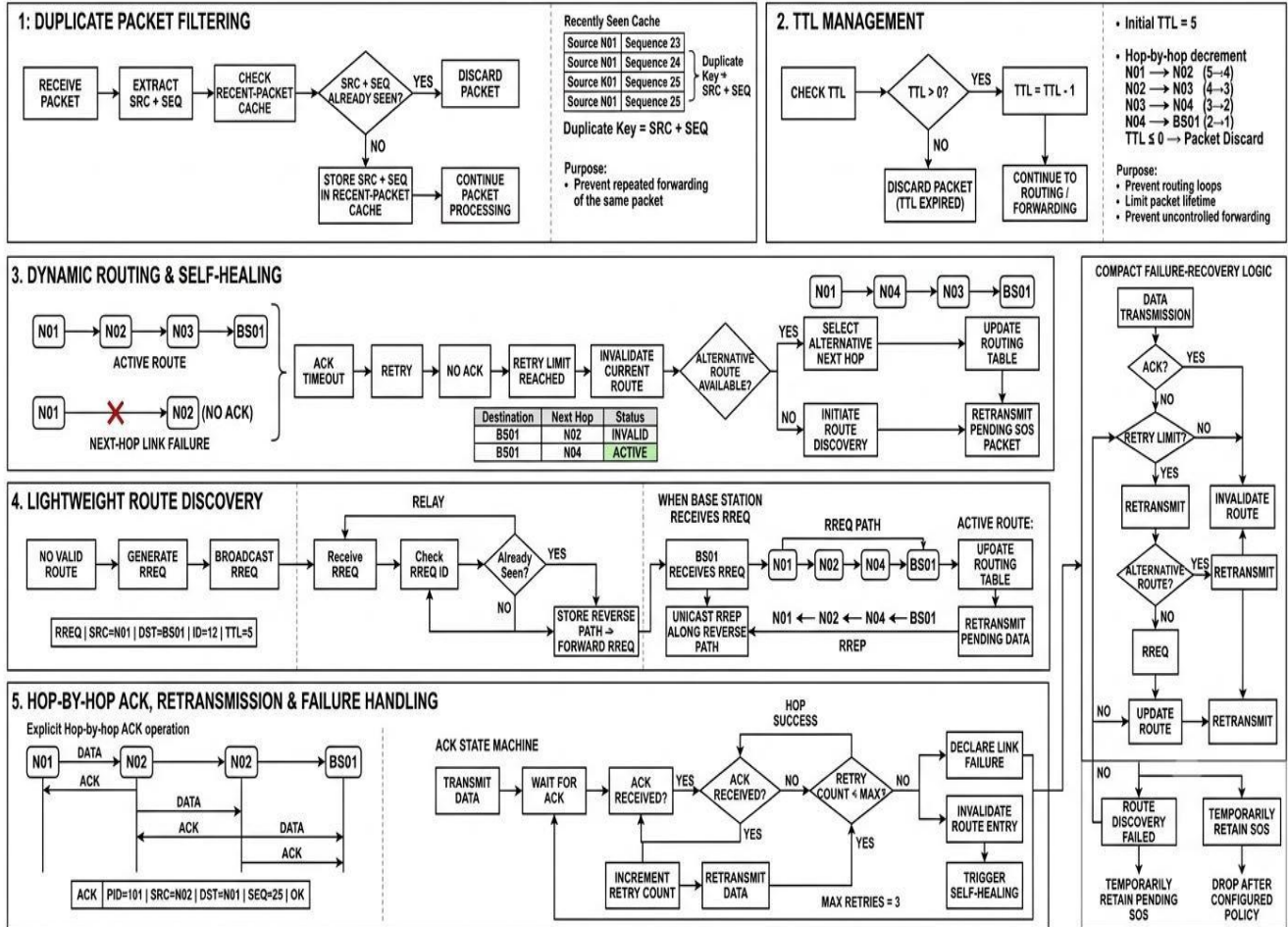
9. Self-Healing / Alternative Route

After detecting a failed next hop, the node checks whether another route is available.



DYNAMIC TERRALINK — COMMUNICATION PROTOCOL: ROUTING, TTL, ACK & SELF-HEALING

LoRa-Based Multi-Hop Emergency Communication



The TerraLink protocol has five core mechanisms:

- 1. Routing:** Find and select the next hop.
- 2. Validation:** Reject corrupted, duplicate, or expired packets.
- 3. Forwarding:** Move the packet hop-by-hop toward the Base Station.
- 4. Reliability:** Use ACK + limited retransmission.
- 5. Self-healing:** Detect failed links, discover an alternative route, update the routing table, and retransmit the pending SOS.

TerraLink combines **LoRa multi-hop forwarding**, **SRC+SEQ duplicate filtering**, **TTL-based loop prevention**, **hop-by-hop ACK/retransmission**, and **lightweight AODV-inspired route recovery** to provide resilient emergency-message delivery.